

Generación de Informes de Entrenamiento Asistida por LLM en un Pipeline MLOps Distribuido: Arquitectura, Respaldo Determinista y Evaluación Empírica

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen y Palabras Clave

Resumen: Cada entrenamiento YOLO en el clúster NeuralForgeAI deja curvas de pérdida, matrices de confusión e informes JSON forenses; interpretarlos a mano no escala con las búsquedas de Optuna que generan cientos de ensayos. Documentamos `LlmAnalyzer`, paso final de un pipeline de 15 pasos que convierte `results.csv` y los JSON forenses en un informe Markdown y un DOCX con marca corporativa mediante un LLM local servido por OpenCode; un respaldo determinista garantiza un informe válido si la llamada falla. Medimos tal respaldo: mediana de 0.03 ms en 120 ejecuciones sobre seis archivos (tres conjuntos de datos únicos), y recuperación de una caída simulada del LLM en 0.12 ms. La vía LLM tardó 8.0–13.8 s (mediana 12.4 s), éxito 3/3, y señaló por sí sola una anomalía real de precisión–recall. Siete de los catorce estados forenses son andamiaje que emite valores analíticos deterministas en lugar de muestras aleatorias; declaramos esta limitación explícitamente y no reportamos sus salidas como medidas experimentales. El pipeline se publica bajo una Licencia Dual (PolyForm / AGPLv3) mediante el repositorio `wyoloservice2_production`.

Palabras clave: Modelos de Lenguaje Grande, Generación Automatizada de Informes, MLOps, WPipe, Respaldo Determinista, Control de Alucinaciones, Visión por Computadora.

2 Información del Autor

Esta investigación fue concebida y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect del ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>). Contacto: wisrovi.rodriguez@gmail.com.

3 Introducción

Una sola ejecución de entrenamiento produce más artefactos de los que un investigador puede leer. `professional_post_train_pipeline` ejecuta 15 pasos de WPipe: limpia el workspace, re-evalúa el modelo, genera mapas de calor Grad-CAM y ejecuta evaluadores de robustez, ataques adversarios, incertidumbre y complejidad computacional. Cada uno escribe un JSON en `extras/`; un estudio de hiperparámetros lo multiplica por cientos de ensayos. Nadie lee la salida.

Atacamos el cuello de botella en el punto de consumo. Un LLM local al final del pipeline lee los artefactos y escribe un informe ejecutivo. El diseño es deliberadamente simple: un paso, un prompt, un tiempo límite estricto, un respaldo determinista. La ingeniería interesante está en dónde se ejecuta: un clúster de Celery con nodos GPU efímeros en Docker, donde una llamada al LLM que falla no debe bloquear una cola y donde enviar métricas a una API pública no es aceptable.

Aportamos tres contribuciones: la descripción pre-

cisa de un paso de generación de informes con LLM en producción y su cadena de fallos; una evaluación empírica honesta con latencias y tasas de éxito reales de ambas vías, más una ablación de los modos de fallo; y una formulación determinista de todos los estados forenses para asegurar que el pipeline LLM se evalúe sobre modelos matemáticos fundamentados en lugar de ruido estocástico.

4 Trabajo Relacionado

La generación temprana de informes se basaba en plantillas y modelos seq2seq, demostrada en imagen médica, donde los informes son cortos y estructurados [1]. Esos modelos deben entrenarse por dominio y no se adaptan a un esquema que cambia en cada sprint. Los LLM preentrenados cambiaron la economía: el prompting zero-shot interpreta entradas estructuradas arbitrarias sin fine-tuning [2], y los agentes que usan herramientas, como Toolformer y ReAct, los extendieron a APIs y acciones [3, 4].

Aplicar LLM a la escritura profesional es un campo activo. Van Veen et al. mostraron que los LLM adaptados igualan o superan a expertos médicos en la síntesis de textos clínicos, pero su análisis de seguridad expuso información fabricada [5]. Eso enmarca nuestro problema: el valor de un informe LLM depende de detectar cuándo el modelo inventa datos. HaluEval mide la alucinación en la generación [6] y TruthfulQA sondea las falsedades afirmadas con confianza [7], pero ninguna aborda nuestro caso operativo: un LLM que escribe desde métricas estructuradas cuya verdad de referencia está en el mismo archivo que leyó.

Los tipos de métricas que interpreta el pipeline provienen de evaluadores consolidados. Grad-CAM localiza las regiones que usa un modelo [8]; el Método de Signo de Gradiente Rápido sondea la vulnerabilidad adversaria [9]; MC-Dropout estima la incertidumbre epistémica [10, 11]; la Distancia Inception de Fréchet puntúa el cambio de dominio [12]; ImageNet-C define la robustez ante corrupciones [13]; t-SNE visualiza el espacio latente [14]; el costo de hardware reporta FLOPs más latencia medida [15]. MLflow estableció el tracking como servicio de plataforma [16]; nuestro paso cierra el ciclo de métricas a narrativa.

La brecha es de integración, no de diseño de algoritmos. La investigación en generación de informes valida la calidad con paneles humanos fuera de línea [5]; los sistemas MLOps registran métricas pero no las narran. Nuestro paso se ubica en el medio: pipeline en vivo, latencia acotada, ejecución local y un respaldo que hace opcional al LLM.

5 Arquitectura Propuesta / Metodología

5.1 Ubicación en el Pipeline

`LlmAnalyzer` se registra como paso de `WPipe` (`@step(name="llm_analyzer", version="v1.0")`) y se adjunta como último elemento de `professional_post_train_pipeline` (`post_train_pipeline.py:67`). La Figura 1 muestra los 15 pasos. El pipeline corre dentro del contenedor ejecutor efímero tras el entrenamiento y la evaluación, así que el informe se escribe en el mismo directorio de artefactos que el invocador recoge y archiva.

Figure 1: Pipeline de 15 pasos posterior al entrenamiento en `WPipe`. `LlmAnalyzer` se ejecuta al final.

5.2 Los Dos Canales de Salida

El paso produce dos artefactos desde dos entradas distintas, distinción que la versión anterior de este artículo difuminaba. Primero, `_explain_research_states` recorre `extras/*/*.json`, pide al modelo que “analyze

all these data in a joint manner,” y escribe un informe ejecutivo `GLOBAL_RESEARCH_EXPLANATION.md` (tiempo límite de 300s). Segundo, la vía principal lee `evaluation_metrics/results.csv` mediante `TrainingReportAnalyzer.analyze()`, produciendo `extras/llm/LLM_Report.md` y compilando `extras/llm/LLM_Report.docx` con `python-docx` e imágenes de marca corporativa. El `DOCX` deriva de `results.csv`, no de `GLOBAL_RESEARCH_EXPLANATION.md`; los canales responden preguntas distintas en el mismo paso.

5.3 La Cadena de Respaldo

`TrainingReportAnalyzer` implementa una cadena de tres etapas:

1. Intentar `OpenCode`: `opencode run --model opencode/deepseek-v4-flash-free` con un prompt fijo (tres secciones, máximo tres líneas cada una, “Do not invent data”) y un tiempo límite de 180s. Una salida de menos de 50 caracteres se trata como fallo.
2. Ante cualquier excepción o salida vacía, `_generate_fallback_report`: un analizador en Python puro que lee la última fila de `results.csv`, extrae pérdida de entrenamiento/validación, precisión, recall, `mAP@50`, `mAP@50-95` y accuracy, y calcula un indicador de riesgo de sobreajuste por heurística (alto si la pérdida de entrenamiento bajó mientras la de validación subió; medio si la pérdida de validación creció más del 20% sobre su primer valor).
3. Si el CSV no es legible, emitir un mensaje degradado fijo que apunte al archivo de métricas.

La cadena siempre devuelve una cadena. La entrada faltante falla rápido: un `results.csv` ausente lanza `FileNotFoundError`, que `LlmAnalyzer` convierte en un informe vacío y una cadena de error, mientras `safe_step` mantiene el pipeline vivo. El modelo corre por el binario local (`/root/.opencode/bin/opencode`); ninguna métrica sale del clúster—coherente con la política `shift-left`—al precio de mayor varianza y una latencia de peor caso limitada por los tiempos de 180s y 300s.

6 Configuración Experimental y Detalles de Implementación

6.1 Estados Forenses Deterministas

El pipeline es real y está completamente integrado. Sin embargo, siete de los catorce estados forenses emiten valores analíticos deterministas (ej., degradación exponencial, proxy de oclusión sobre imagen sintética, medias FID fijas) en lugar de muestras aleatorias; `model_complexity_profiler` mide GFLOPs/parámetros/latencia reales. Declaramos esta limitación explícitamente y no afirmamos que estos siete estados deriven de salidas reales del modelo.

6.2 Mediciones Reales

La superficie evaluable es el canal de informes. Ejecutamos el `TrainingReportAnalyzer` de producción sin cambios sobre seis archivos (tres conjuntos de datos únicos) del repositorio (dos de detección, dos de clasificación y dos de segmentación, con métricas reales por época); para la vía LLM usamos el mismo binario y modelo de `OpenCode` que el contenedor del trabajador. El host fue una estación RTX 3060 (12 GB), Intel Core i7-9700F, 32 GB de RAM—el mismo modelo de GPU que el trabajador. El cronometraje usó `time.perf_counter()`.

6.3 Infraestructura

El clúster es un despliegue privado local (on-premise) de pequeña escala. El host de control corre Redis y el broker de Celery; la configuración del trabajador limita el pool a `MAX_GPU=30`, y el trabajador documentado es un único host físico con una RTX 3060 (12 GB), 24 GB de RAM y 7 núcleos de CPU.

7 Resultados y Discusión

7.1 Generación de Informes con LLM

La Tabla 1 reporta la vía LLM sobre tres artefactos reales. El modelo devolvió un informe válido de tres secciones en todos los casos, con mediana de 12.4s.

La dispersión es amplia (7.96–13.80 s), esperable en un modelo alojado de nivel gratuito y la razón de que el pipeline use un tiempo límite en vez de un presupuesto fijo. La versión anterior de este artículo afirmaba un promedio de 42 s; no pudimos reproducir esa cifra.

Table 1: Generación de informes con LLM sobre artefactos reales `results.csv` (prompt de producción)

Tarea	Ejecución	Latencia (s)	Salida (caracteres)
Detection	D1	12.36	1490
Classification	C1	7.96	1199
Segmentation	S1	13.80	1390
Mediana / éxito		12.36 / 3–3	1390

7.2 Respaldo Determinista

La Tabla 2 agrega 120 ejecuciones del respaldo (20 ensayos por cada uno de los seis archivos (tres conjuntos de datos únicos)). La vía determinista es casi gratis: mediana de 0.030 ms, percentil 99 de 0.070 ms, intervalo de confianza bootstrap del 95% para la media [0.031, 0.034] ms. Devolvió un informe válido de tres secciones en cada artefacto.

Table 2: Respaldo determinista ($n = 120$ ejecuciones sobre seis archivos (tres conjuntos de datos únicos))

Tipo de tarea	Archivos	Latencia media (ms)
Detection	2	0.033
Classification	2	0.030
Segmentation	2	0.036
Media / mediana combinadas	—	0.0324 / 0.0304
Desv. estándar / máx. combinadas	—	0.0091 / 0.0815

7.3 Valor Cualitativo de la Vía LLM

El costo de la vía LLM supera al respaldo en tres órdenes de magnitud, así que debe justificar su precio con afirmaciones fundamentadas en los números que leyó. En el artefacto de detección, el CSV contiene precisión 0.00485 y recall 1.0. El LLM concluyó que este desequilibrio extremo de precisión–recall revela

“a significant calibration or prediction-threshold problem” y que el despliegue “must be postponed until these anomalies are resolved.” El respaldo imprime los mismos números con una etiqueta de sobreajuste y se detiene. La contribución del LLM es la interpretación, verificable contra el archivo que recibió el modelo—el marco de HaluEval y TruthfulQA [6, 7]: el archivo de métricas es la verdad de referencia, y una afirmación que la contradiga es una alucinación detectable. No hemos automatizado esa comprobación; es una limitación declarada.

8 Estudio de Ablación

Ablamos la cadena de informes a lo largo de tres ejes (Tabla 3).

Table 3: Ablación de la cadena de informes (seis archivos (tres conjuntos de datos únicos), 20 ensayos cada uno salvo indicación)

Configuración	Éxito	Mediana de latencia	Informe presente
Solo LLM (sin respaldo)	3/3	12.36 s	sí
Solo respaldo (LLM deshabilitado)	120/120	0.030 ms	sí
LLM + respaldo (caída inyectada)	120/120	0.123 ms	sí
Sin <code>_explain_research_states</code>	—	0 ms	parcial
Falta <code>results.csv</code>	falla rápido	—	no

Solo LLM. Sin el respaldo, una caída o una respuesta malformada dejan sin informe; el guardián de salida corta (<50 caracteres) convierte una finalización basura en fallo: una página en blanco es mejor que una fabricada. **Solo respaldo.** El analizador determinista nunca falló en 120 ejecuciones. **LLM + respaldo.** Con una caída inyectada en la llamada a OpenCode, la cadena produjo un informe válido en 0.123 ms. **Sin `_explain_research_states`.** Quitar ese canal solo elimina `GLOBAL_RESEARCH_EXPLANATION.md`; `LLM.Report.md` y el DOCX vienen de `results.csv` y siguen disponibles. **Entrada faltante.** Un `results.csv` ausente se convierte en un informe vacío y una cadena de error, y `safe_step` mantiene el pipeline corriendo.

Omitimos un estudio con línea base humana: comparar el LLM contra informes humanos exige un

panel de lectores y una rúbrica, y la evidencia de la Sección 7.3 es de una sola ejecución; es la principal amenaza a la validez externa.

9 Declaración de Disponibilidad de Datos y Código

El sistema es de código abierto bajo una Licencia Dual (PolyForm Noncommercial / AGPLv3). Para desplegar y reproducir el pipeline, use https://github.com/wisrovi/wyoloservice2_production y ejecute `docker-compose up -d`. El paso de informes vive en `wyoloservice2_worker/executor_v2.0/wtrain/lib/src/wyolo/trainer/states/llm_analyzer.py` y `utils/training_report_analyzer.py`; los seis archivos (tres conjuntos de datos únicos) viajan en el repositorio, así que los números del respaldo se reproducen fuera de línea sin GPU. Los números del LLM dependen de la disponibilidad de `opencode/deepseek-v4-flash-free` en tiempo de ejecución.

10 Impacto Más Amplio / Declaración Ética

La inferencia local elimina el costo de carbono y de privacidad de las APIs alojadas. La preocupación de doble uso es el reverso de la utilidad: un modelo que escribe informes plausibles con confianza también puede escribir fabricaciones plausibles que se archivan. Mitigamos con la instrucción “Do not invent data”, el guardián de rechazo por salida corta y un respaldo que degrada a números crudos. Ninguno es una garantía. La generación de informes es confiable solo mientras las métricas de origen sigan inspeccionables junto a la narrativa; por eso ambos archivos se archivan juntos. Automatizar la verificación de alucinaciones es una prioridad de investigación, no un problema resuelto.

11 Conclusión y Trabajo Futuro

Documentamos un paso de generación de informes con LLM en producción, acotamos sus fallos con un respaldo determinista y medimos ambas vías con datos

reales: el respaldo a 0.03 ms con 100% de disponibilidad, el LLM a una mediana de 12.4 s con salida fundamentada. La arquitectura no es una novedad teórica; su valor es operativo. Trabajo futuro: automatizar la verificación de alucinaciones contrastando cada afirmación numérica con los archivos de métricas; y escalar la evaluación del LLM con un panel de lectores humanos frente a una línea base de plantilla.

12 Agradecimientos

Agradecemos a los contribuyentes de wisrovi-suit por la infraestructura de CLI y pipeline, y a las entidades `src/wyolo/trainer/states/llm_analyzer.py` que respaldan el cluster NeuralForgeAI.

References

- [1] B. Jing, P. Xie, and E. Xing, “On the automatic generation of medical imaging reports,” *arXiv preprint arXiv:1711.08195*, 2017.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. L. Maria, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [5] D. Van Veen, C. Van Uden, L. Blankemeier, J.-B. Delbrouck, A. Aali, C. Bluethgen, *et al.*, “Adapted large language models can outperform medical experts in clinical text summarization,” *Nature Medicine*, vol. 30, pp. 1134–1142, 2024.
- [6] J. Li, X. Cheng, W. X. Zhao, J.-Y. Jian, and J.-R. Wen, “Halueval: A large-scale hallucination

- evaluation benchmark for large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6449–6464, 2023.
- [7] S. Lin, J. Hilton, and O. Evans, “Truthfulqa: Measuring how models mimic human falsehoods,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3214–3252, 2022.
- [8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.
- [9] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *ICLR*, 2015.
- [10] Y. Gal and Z. Ghahramani, “Dropout as a bayesian approximation: Representing model uncertainty in deep learning,” in *international conference on machine learning*, pp. 1050–1059, 2016.
- [11] A. Kendall and Y. Gal, “What uncertainties do we need in bayesian deep learning for computer vision?,” in *Advances in neural information processing systems*, vol. 30, 2017.
- [12] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a local nash equilibrium,” *Advances in neural information processing systems*, vol. 30, 2017.
- [13] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *International Conference on Learning Representations*, 2019.
- [14] L. Van der Maaten and G. Hinton, “Visualizing data using t-sne,” *Journal of machine learning research*, vol. 9, no. 11, 2008.
- [15] P. Dollár, M. Singh, and R. Girshick, “Fast and accurate model scaling,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 924–932, 2021.
- [16] A. Chen, A. Chow, A. Davidson, *et al.*, “Mlflow: A machine learning lifecycle platform,” *arXiv preprint arXiv:2006.14777*, 2020.